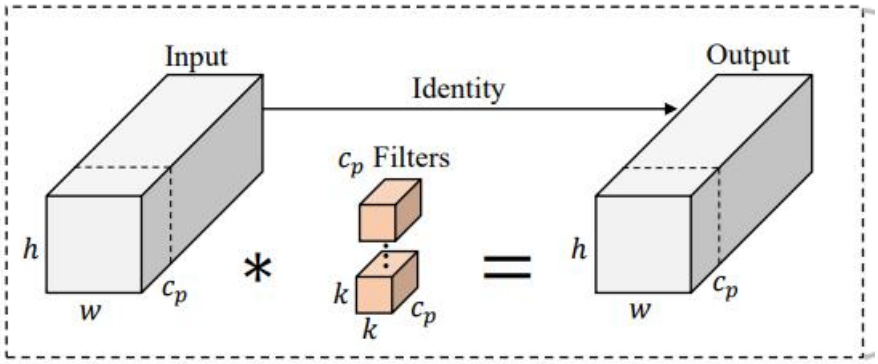
Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks

Jierun Chen[1], Shiu-hong Kao[1], Hao He[1]
Weipeng Zhuo[1], Song Wen[2], Chul-Ho Lee[3], S.-H. Gary Chan[1]
[1]HKUST, [2]Rutgers University, [3]Texas State University



我们以 CVPR2023 部分卷积这篇文章作为开篇。

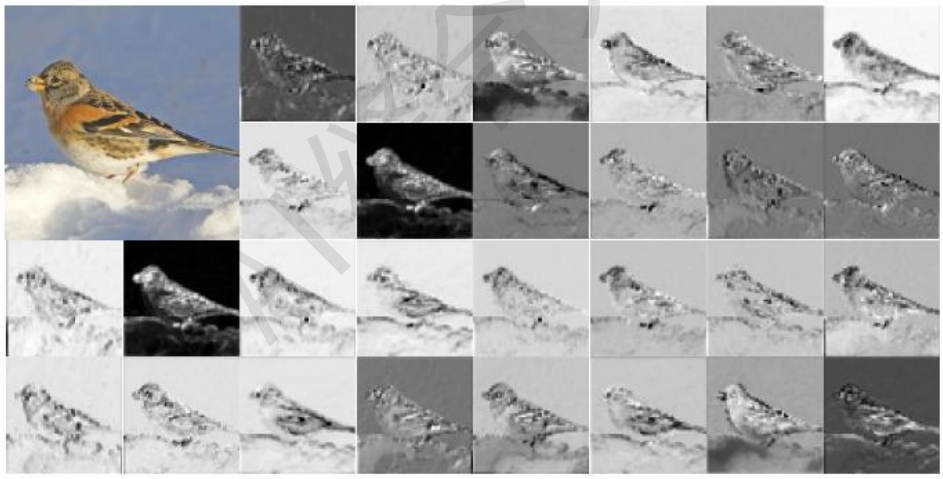


Figure 3. Visualization of feature maps in an intermediate layer of a pre-trained ResNet50, with the top-left image as the input. Qualitatively, we can see the high redundancies across different channels.

Figure 3 展示了特征图不同通道展现了极高的冗余，为此，本文就对一部分通道做卷积，其他的通道保持不变。

比如说，我们的部分卷积 PConv 只对 1/4 的通道进行卷积操作，其余 3/4 通道不做任何处理，这样我们的 FLOPS 降低为常规卷积的 1/16，内存访问降为常

规卷积的 1/4。

```python
class Partial_conv3(nn.Module):

    def __init__(self, dim, n_div, forward):
        super().__init__()
        self.dim_conv3 = dim // n_div
        self.dim_untouched = dim - self.dim_conv3
        self.partial_conv3 = nn.Conv2d(self.dim_conv3, self.dim_conv3, 3, 1, 1, bias=False)

        if forward == 'slicing':
            self.forward = self.forward_slicing
        elif forward == 'split_cat':
            self.forward = self.forward_split_cat
        else:
            raise NotImplementedError

    def forward_slicing(self, x: Tensor) -> Tensor:
        # only for inference
        x = x.clone()   # !!! Keep the original input intact for the residual connection later
        x[:, :self.dim_conv3, :, :] = self.partial_conv3(x[:, :self.dim_conv3, :, :])

        return x

    def forward_split_cat(self, x: Tensor) -> Tensor:
        # for training/inference
        x1, x2 = torch.split(x, [self.dim_conv3, self.dim_untouched], dim=1)
        x1 = self.partial_conv3(x1)
        x = torch.cat((x1, x2), 1)

        return x
```

可以看到，代码的话，也非常简单，就是只对一部分通道进行卷积处理，如通道的 1/4、1/2 等等，其余部分不做操作。

一个即插即用的模块 ：比如说 cvpr2023 这个部分卷积

你的第一想法是什么，缝合到模型里，用于替换普通卷积，或者和其他模块做串并联。

我们看一下这篇文章 ACM MM 2023 的文章：Enhancing Real-Time Super Resolution with Partial Convolution and Efficient Variance Attention。这篇文章名字上就带了部分卷积 Partial Convolution。

这是一篇关于超分辨率重建的文章。

AI缝合大王

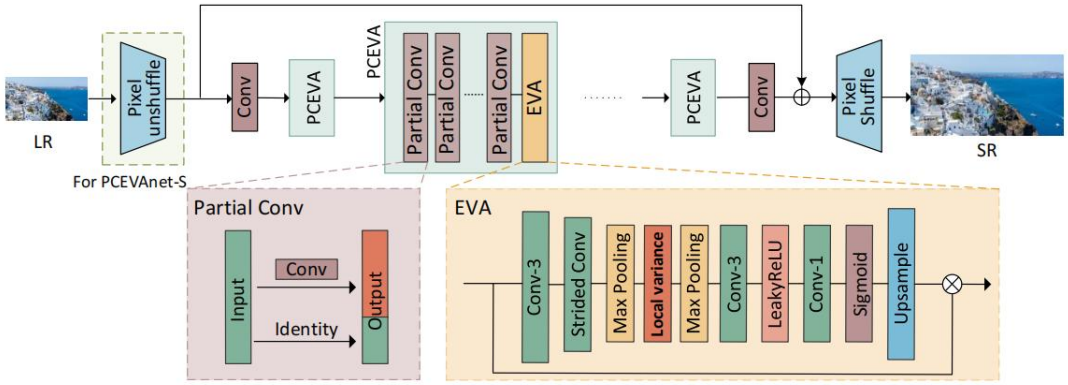


Figure 2: Network architecture for Partial Convolution and Variance Attention (PCEVA) Network. The wisteria box represents the structure of partial convolution and the orange box represents the structure of Efficient Variance Attention (EVA).

Scale	Model	#Params (K)	#FLOPs (G)	Latency (ms)	Set5 PSNR / SSIM	Set14 PSNR / SSIM	B100 PSNR / SSIM	Urban100 PSNR / SSIM	Manga109 PSNR / SSIM
×2	Bicubic	–	–	–	33.66 / 0.9299	30.24 / 0.8688	29.56 / 0.8431	26.88 / 0.8403	30.80 / 0.9339
	SRCNN [11]	69.3	143.4	14.7	36.66 / 0.9542	32.45 / 0.9067	31.36 / 0.8879	29.50 / 0.8946	35.60 / 0.9663
	FSRCNN [12]	24.7	33.9	17.8	37.00 / 0.9558	32.63 / 0.9088	32.63 / 0.9088	29.88 / 0.9020	36.67 / 0.9710
	SESR-M3 [4]	13.0	6.7	2.8	37.21 / 0.9577	32.70 / 0.9100	31.56 / 0.8920	29.92 / 0.9034	36.47 / 0.9717
	ECBSR-M4C16[49]	11.5	5.9	2.7	**37.33 / 0.9593**	32.81 / **0.9129**	31.66 / **0.8961**	30.31 / 0.9091	-
	PCEVAnet-S	147.1	18.3	2.6	37.29 / 0.9581	**32.92** / 0.9119	**31.68** / 0.8935	**30.48 / 0.9106**	**37.42 / 0.9737**
	SESR-XL [4]	113.8	59.0	7.7	37.77 / 0.9601	33.24 / 0.9145	31.99 / 0.8976	31.16 / 0.9184	38.01 / **0.9759**
	ECBSR-M10C32[49]	97.2	50.2	7.3	37.76 / **0.9609**	33.26 / 0.9146	**32.04 / 0.8986**	31.25 / 0.9190	-
	PCEVAnet-M	68.7	27.6	6.6	**37.83** / 0.9603	**33.39 / 0.9157**	32.02 / 0.8979	**31.39 / 0.9202**	**38.18** / 0.9758
	CARN-M [1]	257.0	205.2	70.6	37.53 / 0.9583	33.26 / 0.9141	31.92 / 0.8960	31.23 / 0.9193	-
	CARN [1]	964.2	501.4	53.5	37.88 / 0.9601	33.54 / 0.9173	32.14 / 0.8990	31.96 / 0.9267	-
	ECBSR-M16C64[49]	600.6	310.8	24.1	37.90 / 0.9615	33.34 / 0.9178	32.10 / 0.9018	31.71 / 0.9250	-
	IMDN [20]	873.2	451.4	47.3	38.00 / 0.9605	33.63 / 0.9177	32.19 / 0.8996	32.17 / 0.9283	**38.88** / 0.9774
	RFDN [28]	417.2	205.5	37.4	38.05 / 0.9606	33.68 / 0.9184	32.16 / 0.8994	32.12 / 0.9278	38.88 / 0.9773
	PCEVAnet-L	375.8	182.2	26.9	**38.17 / 0.9618**	**33.79 / 0.9196**	**32.24 / 0.9008**	**32.18 / 0.9291**	38.81 / **0.9775**
×4	Bicubic	–	–	–	28.42 / 0.8104	26.00 / 0.7027	25.96 / 0.6675	23.14 / 0.6577	24.89 / 0.7866
	SRCNN [11]	69.3	573.6	59.4	30.48 / 0.8626	27.50 / 0.7513	26.90 / 0.7101	24.52 / 0.7221	27.58 / 0.8555
	FSRCNN [12]	24.7	118.5	5.8	30.72 / 0.8660	27.61 / 0.7550	26.98 / 0.7150	24.62 / 0.7280	27.90 / 0.8610
	SESR-M3 [4]	27.4	14.18	3.4	30.75 / 0.8714	27.62 / 0.7579	27.00 / 0.7166	24.61 / 0.7304	24.41 / 0.7263
	ECBSR-M4C16[49]	16.8	8.6	3.5	**31.04 / 0.8805**	27.78 / **0.7693**	27.09 / **0.7283**	24.79 / 0.7422	-
	PCEVAnet-S	230.7	29.1	3.3	30.97 / 0.8757	**27.87** / 0.7636	**27.10** / 0.7203	**24.90 / 0.7422**	**28.43 / 0.8751**
	SESR-XL [4]	142.6	73.9	8.7	31.54 / 0.8866	28.12 / 0.7712	27.31 / 0.7277	24.80 / 0.7389	29.65 / 0.8189
	ECBSR-M10C32[49]	107.6	55.6	8.1	**31.66 / 0.8911**	28.15 / **0.7776**	27.34 / **0.7363**	25.41 / **0.7653**	-
	PCEVAnet-M	79.1	33.0	7.4	31.58 / 0.8867	**28.25** / 0.7718	**27.36** / 0.7279	**25.48** / 0.7618	**29.47 / 0.8925**
	CARN-M [1]	294.2	292.4	95.3	31.92 / 0.8903	28.42 / 0.7762	27.44 / 0.7304	25.62 / 0.7694	
	CARN [1]	1111.9	817.9	82.1	32.16 / 0.8943	28.59 / 0.7810	27.58 / 0.7355	26.03 / 0.7830	-
	ECBSR-M16C64[49]	621.4	321.6	24.9	31.92 / 0.8946	28.34 / 0.7817	27.48 / **0.7393**	25.81 / 0.7773	-
	IMDN [20]	893.9	462.2	47.9	32.21 / 0.8948	28.58 / 0.7811	27.56 / 0.7353	26.04 / 0.7838	30.45 / 0.9075
	RFDN [28]	433.4	213.9	37.7	32.24 / 0.8952	28.61 / 0.7819	27.57 / 0.7360	26.11 / 0.7858	**30.58 / 0.9089**
	PCEVAnet-L	401.8	195.6	28.2	**32.27 / 0.8963**	**28.68 / 0.7823**	**27.67** / 0.7383	**26.30 / 0.7879**	30.57 / 0.9078

Table 2: Performance comparison of different SR models on five benchmark datasets. PSNR / SSIM on the Y channel is reported on each dataset. Inference Latency is tested on RTX 3090 (24GB) using FP16 precision. The FLOPs and inference latency are measured under the setting of generating 2560×1440 image.

可以看到这篇文章侧重于轻量化，直接使用了部分卷积 PConv，在此基础上，又加入了一个自己提出的模块 EVA。不过可以看到，部分卷积在这里占了很大一部分。

从这篇文章上，我们可以看到，简单的应用这些即插即用模块，做轻量化或者效果有提升，简单的使用，确实也可行，是可以发不错的文章的。

除此之外，还有什么可以做？

你能想到什么？

如何利用其创新点？

（建议思考一下，再看下面的文章）

AI缝合大王

给大家分享这篇 CVPR2024 的文章： SHViT: Single-Head Vision Transformer with Memory Efficient Macro Design。

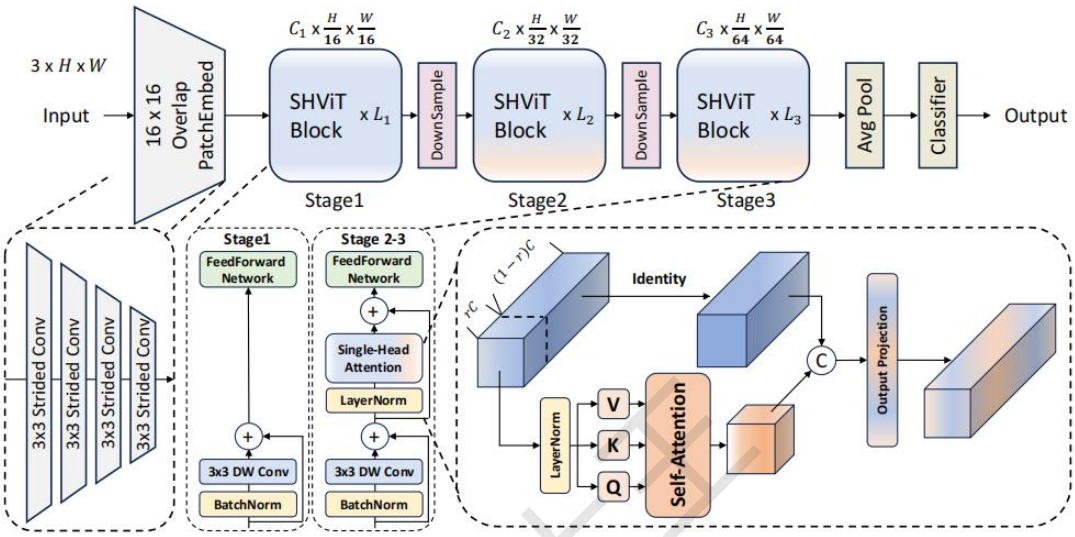
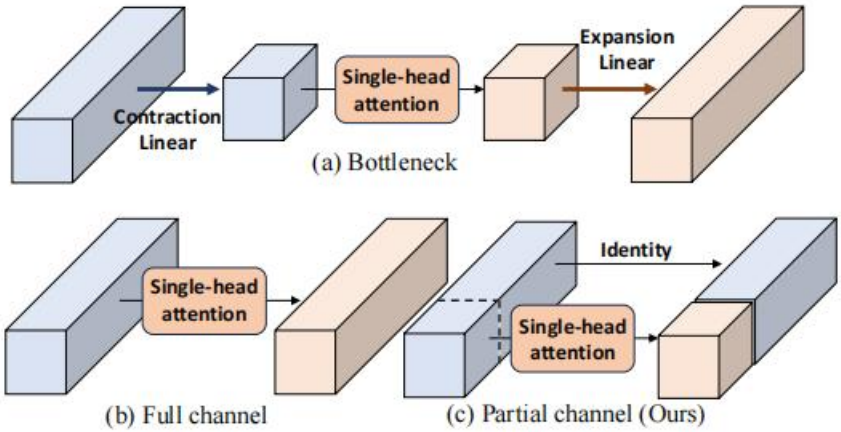


Figure 5. **Overview of Single-Head Vision Transformer (SHViT).** The model starts with a 16×16 overlapping patch embedding layer and uses single-head attention layers in the latter stages to efficiently compute global dependencies. See text for details.

看到这个，你又会有什么思考

AI缝合大王



Single-Head Attention design	Params (M)	CPU_{ONNX} (images/s)	GPU (images/s)	Top-1 (%)
(a) : Bottleneck	10.6	802	27195	73.6
(b) : Full channel	10.4	724	26432	75.1
(c) : Partial channel (Ours)	11.4	**951**	**26878**	**75.2**

Figure 6. **Comparison of Single-head attention designs. (a)** replaces convolution with single-head attention in ResNet's bottleneck block [4]. The contraction ratio is equal to the partial ratio in (c). **(b)** uses full channels for single-head attention modules. All models are configured to have similar speeds. Our partial channel approach has the best speed-accuracy tradeoff.

卷积存在冗余，可以使用一部分通道，同理，注意力机制是不是也可以！ 那是不是所有模块，都有这个可能性!!!

就是说，这个部分卷积的思想，

可以应用到所有模块？？？

AI缝合大王

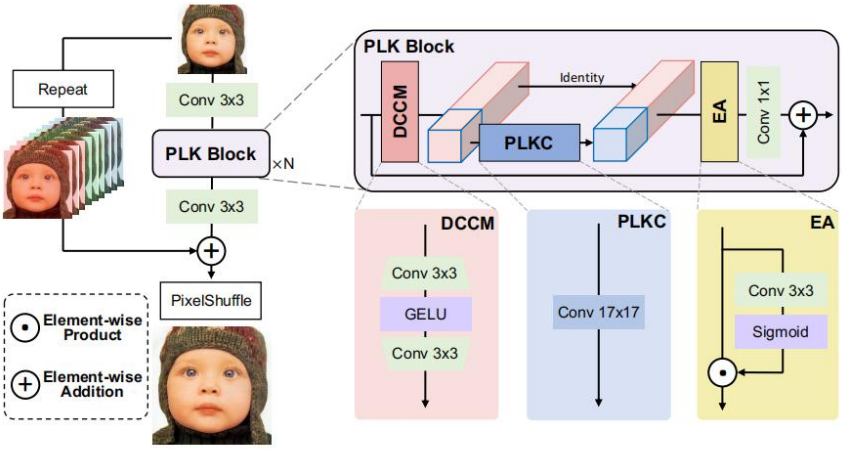


Figure 3. Overview of our architecture. The PLK (Partial Large Kernel) Block, the main block of PLKSR, consists of three modules: DCCM (Double Conv Channel Mixer), PLKC (Partial Large Kernel Conv), and EA (Element-wise Attention).

给大家分享这篇文章（Arxiv2024）Partial Large Kernel CNNs for Efficient Super-Resolution。

这篇论文就是引用到大核上面，使用大核的一部分，

看到这个你又有什么想法。

还在等什么吗。

是不是可以考虑，把这个“部分”思想，拓展到所有的模块中，自己的任务里！

自己的模块，Partial xxx，是不是就来了。

AI缝合大王